

Solving Problems by Searching

COMP 469: Introduction to Artificial Intelligence

CALIFORNIA STATE UNIVERSITY CHANNEL ISLANDS

3.1 Problem-Solving Agents

An agent can **look ahead** to find a sequence of actions that achieves its goal. This process is called **Search**.

The 4-Phase Process

- 🕒 **Goal Formulation:** Define objectives.
- 📖 **Problem Formulation:** Model the world.
- 🔍 **Search:** Simulate action sequences.
- ▶ **Execution:** Apply the solution.

Atomic Representations

States are considered as "wholes" with no visible internal structure. This differentiates search from planning (Chapter 11).

"In a fully observable environment, a solution is a fixed sequence of actions (open-loop)."

Formal Problem Definition



State Space

The set of all possible states the environment can be in (e.g., all Romanian cities).



Actions & Model

ACTIONS(s): Legal moves.
RESULT(s, a): Transition model.



Cost & Goal

GOAL-TEST(s): Recognition logic.
STEP-COST(s, a, s'): Numeric value.

$$\text{Solution} = \{ a_1, a_2, \dots, a_n \} \rightarrow \text{Path Cost} = \sum_{i=1}^n c(s_{i-1}, a_i, s_i)$$

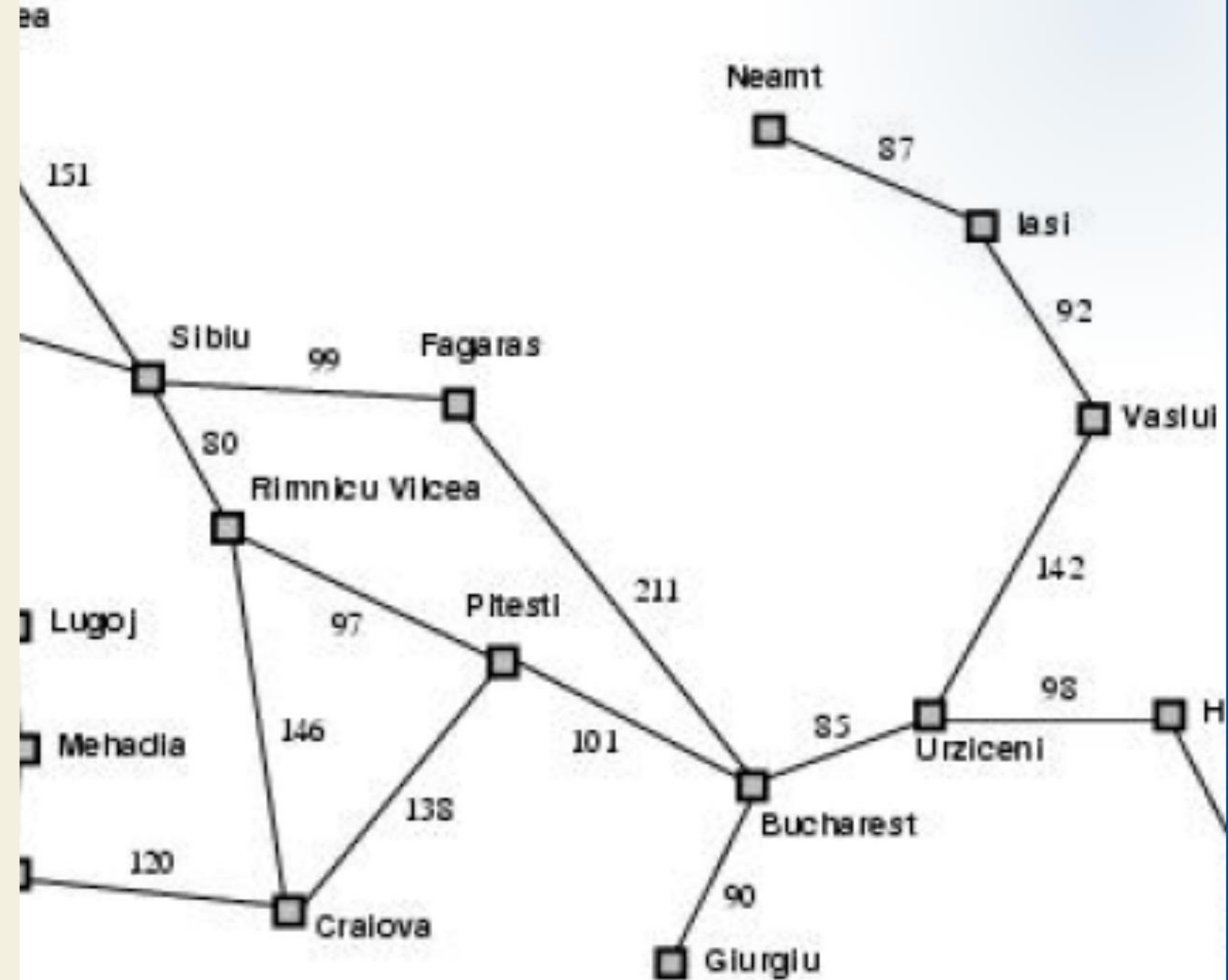
Case Study: Romania

Abstraction in Action

We ignore radio stations, weather, and scenery to focus on "Cities" and "Roads".

- **State:** Current city (e.g., Arad)
- **Actions:** Drive to neighboring city
- **Goal:** Be in Bucharest
- **Path Cost:** Sum of distances in miles

"Abstraction is useful if the resulting solution is easier to execute than the original problem."



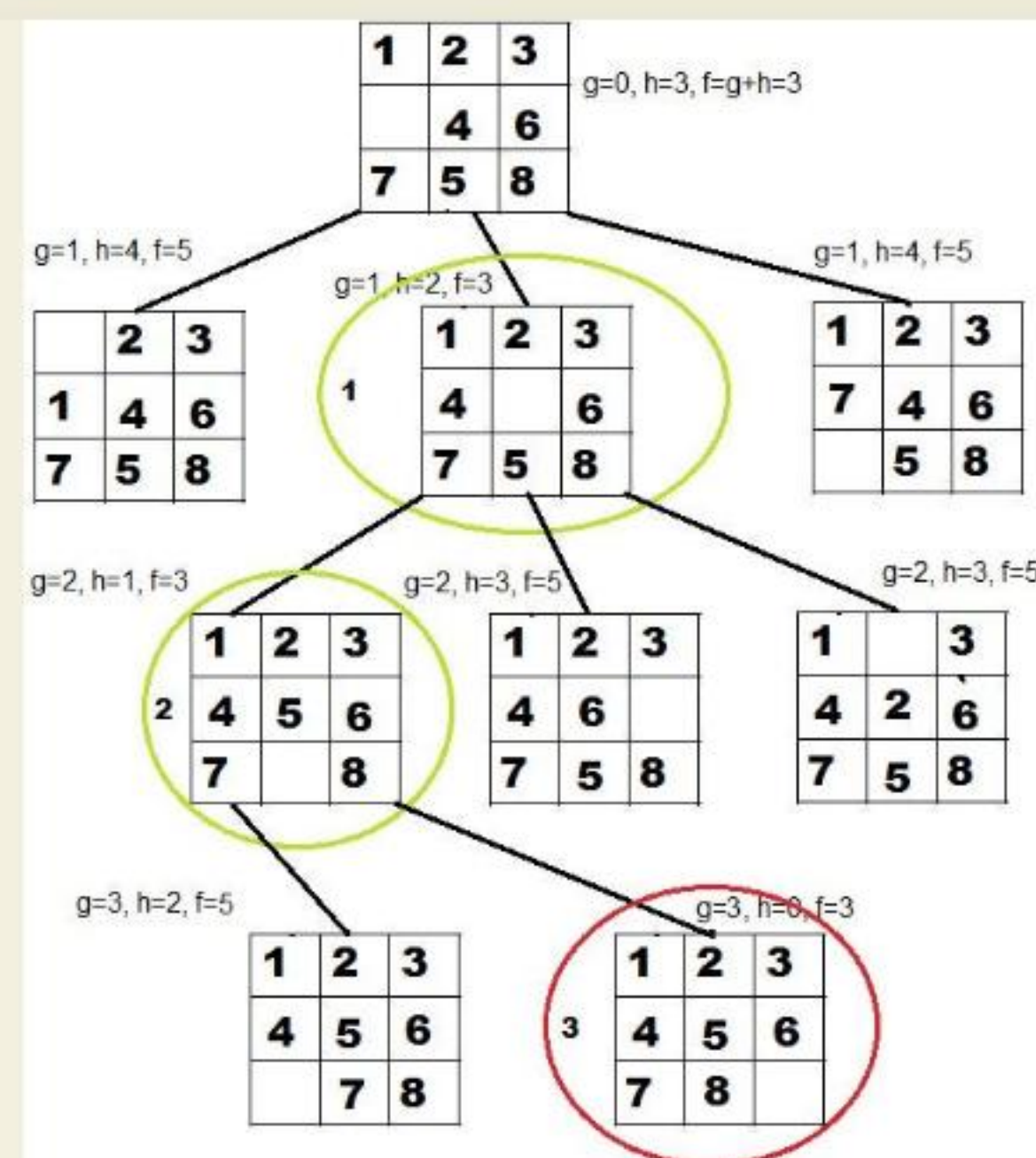
ia. The numbers connecting cities represent distance

Standardized Problems

The 8-Puzzle

A sliding-tile puzzle on a 3x3 grid. The blank space "moves".

- **States:** Location of 8 tiles + blank.
- **Initial:** Random configuration.
- **Complexity:** 181,440 states. (15-puzzle: 10 trillion).

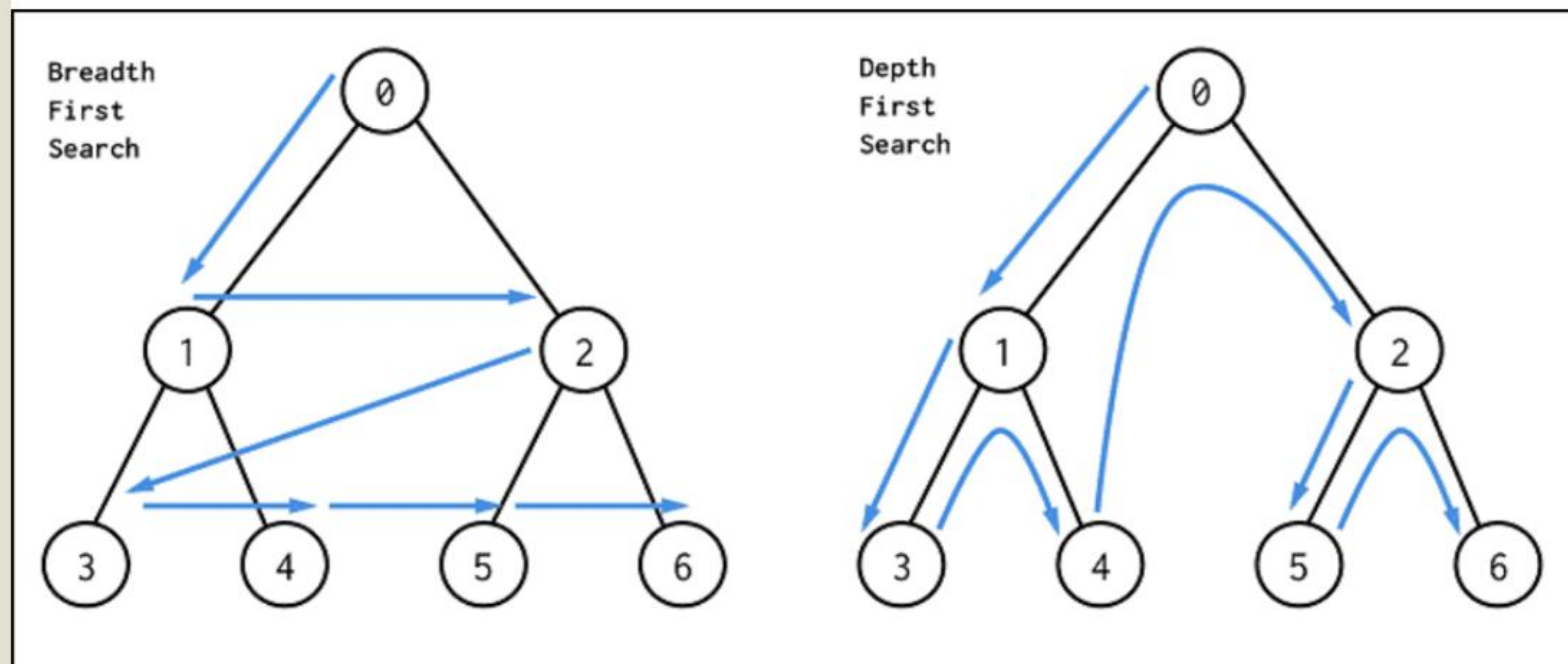


The Grid World

Commonly used for Robotics and Vacuum-Cleaner agents.

Infinite State Spaces: Problems like Knuth's "4" (applying sqrt, floor, factorial) lead to infinite branching trees, requiring systematic search to ensure completeness.

Search Infrastructure



The Data Structure: Node

A node is NOT a state. A node is a book-keeping structure containing:

- **STATE:** The configuration.
- **PARENT:** Pointer to previous node.
- **ACTION:** Applied to parent.
- **PATH-COST:** $g(n)$, total cost from root.

Frontier: The set of all leaf nodes available for expansion.

Best-First-Search Algorithm

```
function BEST-FIRST-SEARCH(problem, f) returns a solution node or failure
  node ← NODE(STATE=problem.INITIAL)
  frontier ← a priority queue ordered by f, with node as element
  reached ← a lookup table (hash map) with state:node
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    if problem.IS-GOAL(node.STATE) then return node
    for each child in EXPAND(problem, node) do
      s ← child.STATE
      if s is not in reached or child.PATH-COST < reached[s].PATH-COST then
        reached[s] ← child
        add child to frontier
  return failure
```

Varying the evaluation function **f(n)** gives us different algorithms.

Breadth-First Search

Logic: Layer by Layer

Expands the shallowest nodes first. Uses a **FIFO Queue**.

Completeness: Yes (if b is finite).

Optimality: Yes, if all step costs are equal.

Complexity & The Memory Wall

Search tree grows exponentially:

$$\text{Time/Space Complexity} = O(b^d)$$

At $b=10$, $d=10$, we need **10 Terabytes** of memory. Memory is usually the bottleneck for BFS.

Uniform-Cost Search

Evaluating Path Cost

Expand node with the lowest path cost $g(n)$.

Goal Test: Must be performed when a node is *popped/expanded*, not when generated.

Optimal: Always finds the cheapest path.

Romania: Sibiu to Bucharest

Fagaras is reached first (cost 99), but Pitesti (reached later) leads to a cheaper path (278 vs 310). UCS correctly selects Pitesti because it expands in order of total $g(n)$.

Depth-First Variants

Depth-First Search (DFS)

Expands deepest node first (LIFO/Stack). **Space Complexity: $O(bm)$** . Much more memory efficient than BFS.

Risk: Infinite loops in cyclic graphs.

Iterative Deepening (IDS)

Calls Depth-Limited Search with limits 0, 1, 2... until goal found.

- **Best of both:** $O(bd)$ space (like DFS) and completeness (like BFS).
- **Wasteful?** Not really. Bottom layer dominates the node count.

Section 3.5: Informed Search

Using domain-specific hints (Heuristics) to guide the agent.

Heuristic Functions $h(n)$

$h(n)$ = estimated cost of the cheapest path from node n to a goal state.

Examples

- **Route Finding:** Straight-line distance (Euclidean).
- **8-Puzzle:** Number of misplaced tiles.
- **8-Puzzle:** Manhattan Distance (City-block).

Domain Knowledge

Unlike $g(n)$ which is measured, $h(n)$ is an *estimate* based on properties of the state. It "points" the search in the right direction.

$h(goal)$ must be 0.

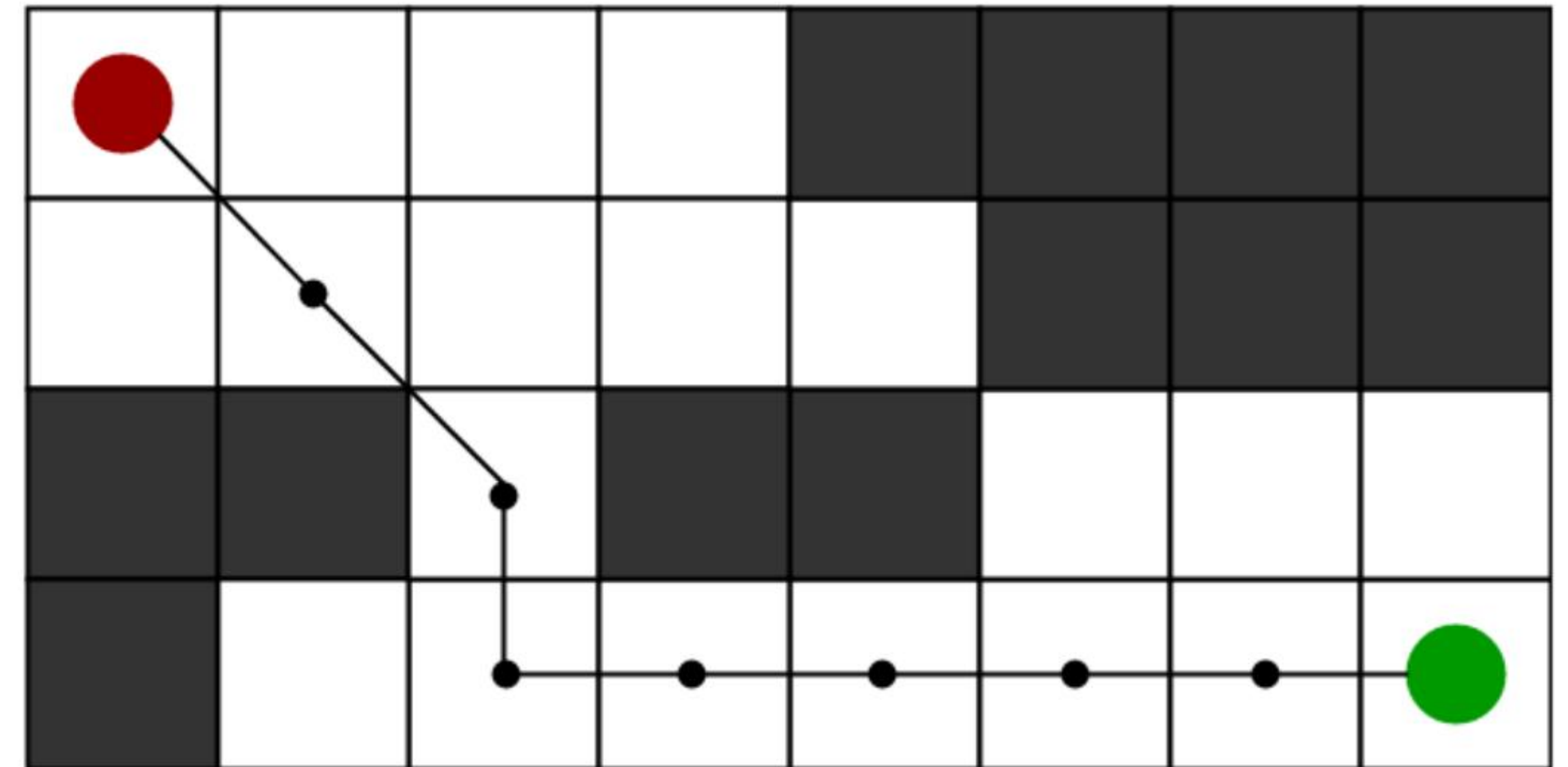
Greedy Best-First Search

Evaluation: $f(n) = h(n)$

Expands the node that *appears* to be closest to the goal.

⚠ **Myopic:** It ignores the cost already incurred to reach the node.

✗ **Not Optimal:** Can lead to suboptimal paths if the heuristic is misleading.



A* Search: Optimal Guarantees

Combines historical cost and future estimate:

$$f(n) = g(n) + h(n)$$

Admissibility

A heuristic is **admissible** if it *never overestimates* the cost to reach the goal. It is optimistic.

A* is optimal if $h(n)$ is admissible.

Consistency

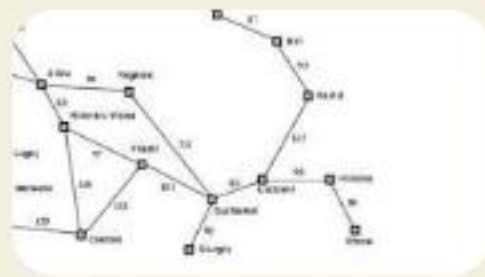
$h(n)$ is **consistent** (monotone) if for every node n and successor n' , the estimated cost doesn't decrease more than the actual step cost.

Algorithm Comparison Summary

Criterion	BFS	UCS	DFS	IDS	A*
Complete?	Yes	Yes	No	Yes	Yes
Optimal?	Unit Cost	Yes	No	Unit Cost	Yes
Time	$O(b^d)$	$O(b^{C^*/\epsilon})$	$O(b^m)$	$O(b^d)$	$O(b^d)^*$
Space	$O(b^d)$	$O(b^{C^*/\epsilon})$	$O(bm)$	$O(bd)$	$O(b^d)^*$

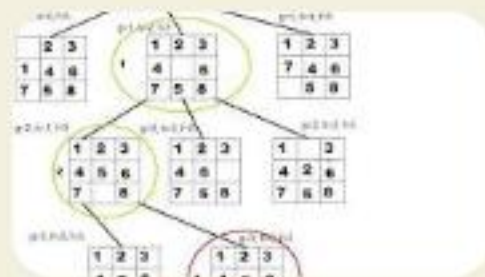
** A* complexity depends heavily on the accuracy of $h(n)$.*

Image Sources



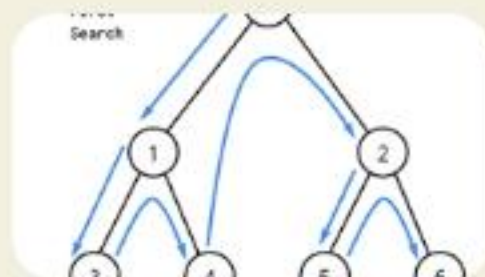
<https://media.cheggcdn.com/media/371/371b5b59-ebc5-477a-81eb-c4038fd4ac17/phpubTIVu>

Source: www.chegg.com



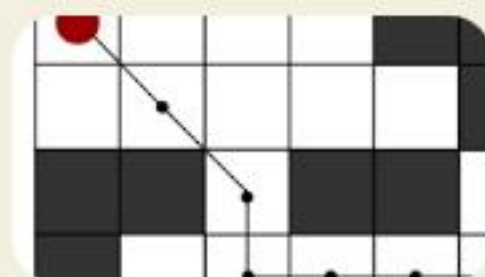
https://miro.medium.com/1*Bz0qSCPmRb509DRYpYEEkQ.jpeg

Source: blog.goodaudience.com



https://miro.medium.com/1*fjiil9FYXuT-u-rt2UfbjA.png

Source: ashley-gaskins.medium.com



https://media.geeksforgeeks.org/wp-content/uploads/a_-search-algorithm-1.png

Source: www.geeksforgeeks.org